ECE60827 CUDA Assignment 3
Jerry Liu, 3/25/2023

Convolution:

Overview:

Initially, the parallelization strategy for convolution was to launch an initial thread for every output element, and then have each of these parent threads launch one additional child kernel. Each thread in the child kernel would handle a single multiplication of one filter weight and one input element, and together they would compute all of the partial products needed for their respective parent kernels:

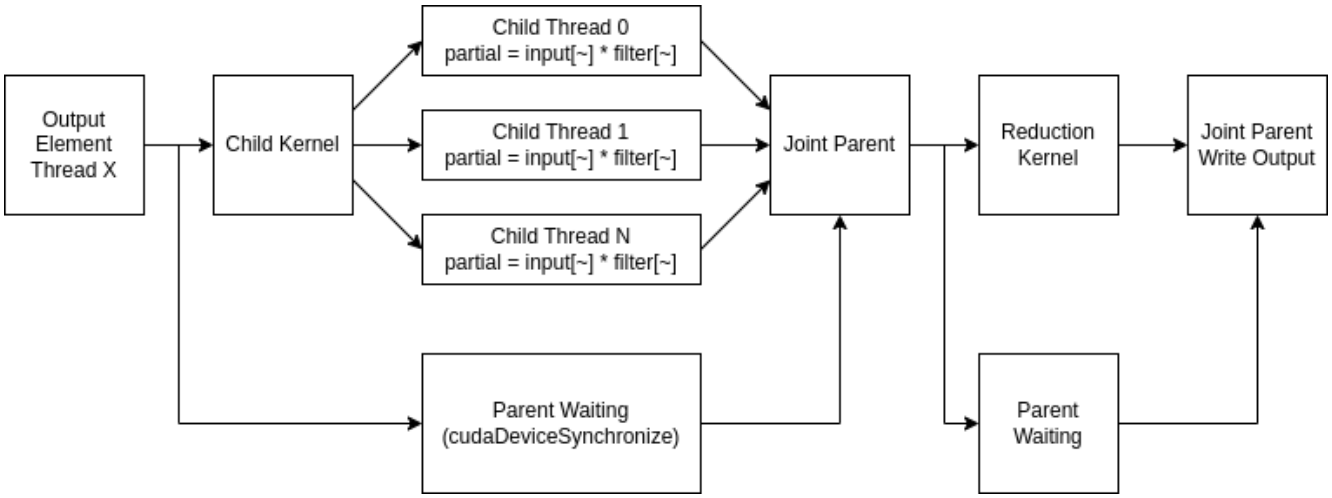


Figure 1: Concept Dynamic Parallelism Logic Flow

The benefit of this approach was that, instead of computing and summing the partial products sequentially, they could be done in parallel with the help of a parallel reduction function/kernel. However, because dynamic parallelism does not allow the sharing of registers or shared memory between kernels, the total sharers per element of data would be one because each child thread would load and use data for itself only. It should also be noted that a similar approach can also be done without dynamic parallelism by spawning more threads initially. Because of these limitations/overheads, the overall strategy was changed to enable more efficient shared memory utilization. The implemented strategy for convolution still assigns one thread to each output element (x, y), with the z dimension *mainly* mapping to the output channels. Each thread first loads the filter weights for the given output channel into a shared tile. Then, the input elements are loaded from global memory, and the partial sum is computed used the previously loaded weights. Note that constant memory was not used for the input tensor because the access pattern is not suited for broadcast, and shared memory was not used because the number of sharers (given the thread block layout) is low/zero. The output is then stored into global memory.

ECE60827 CUDA Assignment 3
Jerry Liu, 3/25/2023

One key optimization implemented is parallel computation of each output value. Since each output element is made up of R * S * C filter weights, the available shared memory per thread block is often too small to store the entire tensor at once. This means that computation of the output needs to be broken up into several steps, with each step/section computing a partial sum of the total output value. Instead running each "step" sequentially in one thread (load, sum, repeat), the optimized convolution spawns extra thread blocks along the z dimension to account for the extra steps needed. Thus, the final layout of the threads are as follows:

{x, y, z} ~ { [0 – oShape.width), [0 - oShape.height), [0 - (oShape.channel * X steps)) }

In addition to this parallel optimization, the thread block size was adjusted to achieve the highest calculated occupancy as reported by Nsight Compute (16x16). Finally, the loading of the shared tile is optimized for memory coalescing as was done perviously:

Example optimized load pattern:
Thread 0: input[0], input[16], input[32] …
Thread 1: input[1], input[17], input[33] …
…

Test Results (3 run average):

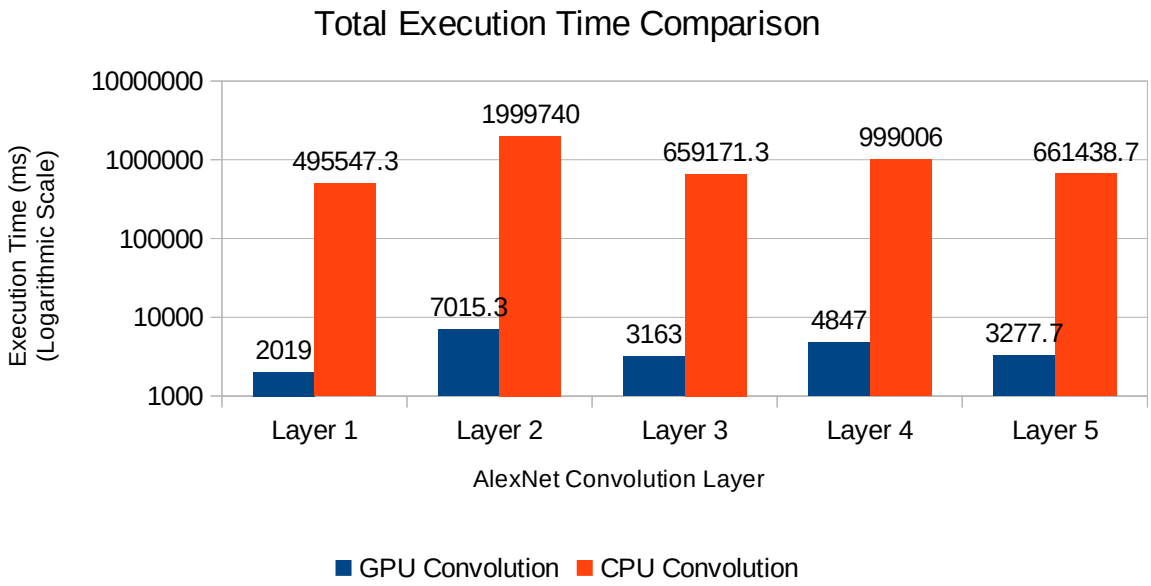


Figure 2: All execution times were measured using std:chrono at the call of the actual algorithm (i.e. convLayer_gpu). Tensor dimensions for each layer taken from AlexNet.

ECE60827 CUDA Assignment 3
Jerry Liu, 3/25/2023

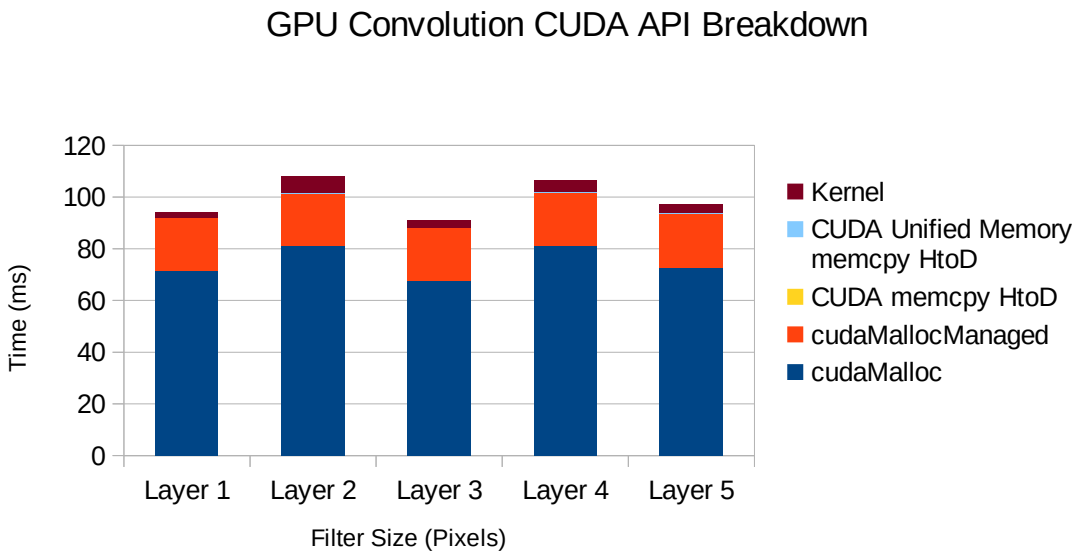


Figure 3: Breakdown of GPU convolution API calls using command "nsys nvprof ./lab3" Non-relevant API calls are excluded (i.e. cudaLibraryLoadData) for clarity.

Observations:

Overall, the GPU implementation is much faster than the CPU implementation, with the GPU version being about 230 times faster on average. At the size of the dataset tested, memory operations still dominate the total execution time, with the kernel taking up less than a quarter of the total GPU execution time. This may change as the dataset increases, but the overall trend is unclear with the tests that have been run.

General Purpose Matrix Multiply:

Overview:

The use of dynamic parallelism was also considered for GEMM, but it was ultimately dropped because the overhead of additional kernel launches. Similar functionality can be achieved without the use of dynamic parallelism, so the added overhead would have resulted in worse overall performance. Instead, the strategy is nearly identical to convolution, with the x and y dimensions used to handle elements in one channel, and the z dimension used to handle subtiles and channels. Each thread block first loads both the a and b matrix into shared memory, and then each thread uses these shared values to compute their respective outputs.

The optimizations for GEMM are also remarkably similar to convolution since they are similar operations. Once again, the use of parallel output computation is used to compute subtiles in parallel, but this time a sequential version was also developed in order to measure the performance uplift. For shared memory optimization, loading of both shared subtiles are orchestrated such that the thread access pattern is better for

memory coalescing, and the shared memory block is over-sized to help reduce the total number bank conflicts. Finally, the thread block size (32x32) was adjusted to increase the achieved occupancy as reported by Nsight Compute.

Test Results (3 run average):

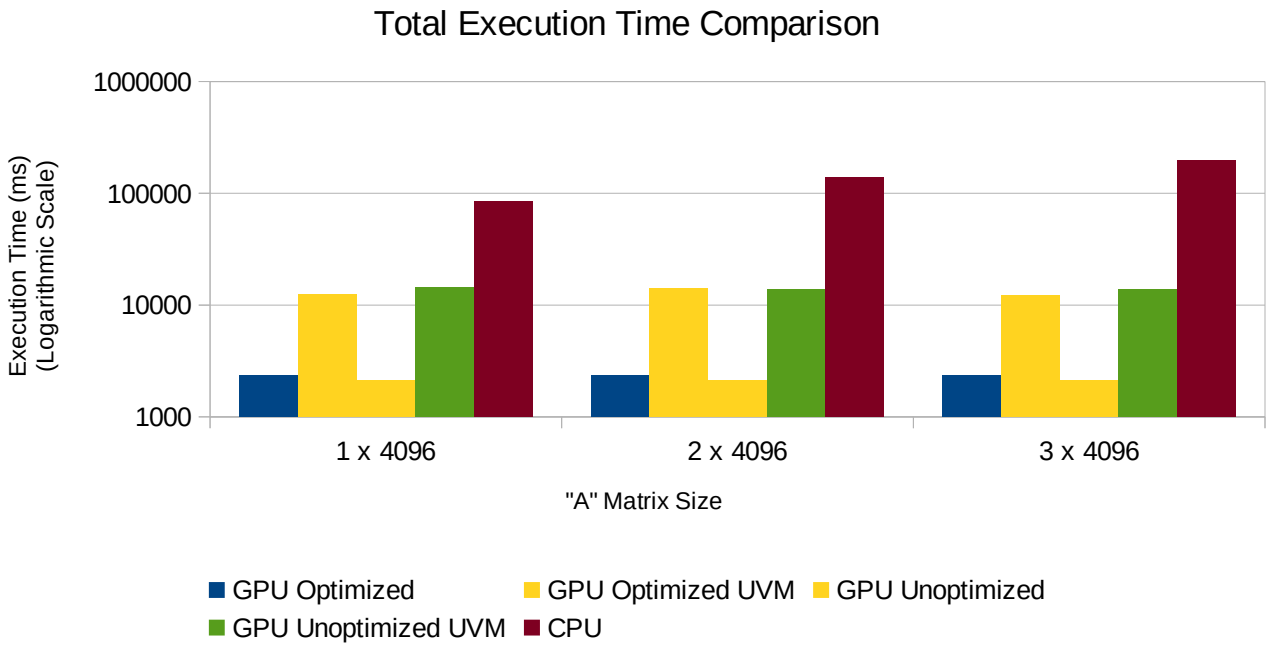


*Figure 4: All execution times were measured using std:chrono at the call of the actual algorithm (i.e. gemmLayer_gpu). Results collected from a (N x 4096) * (4096 x 4096) matrix multiply.*

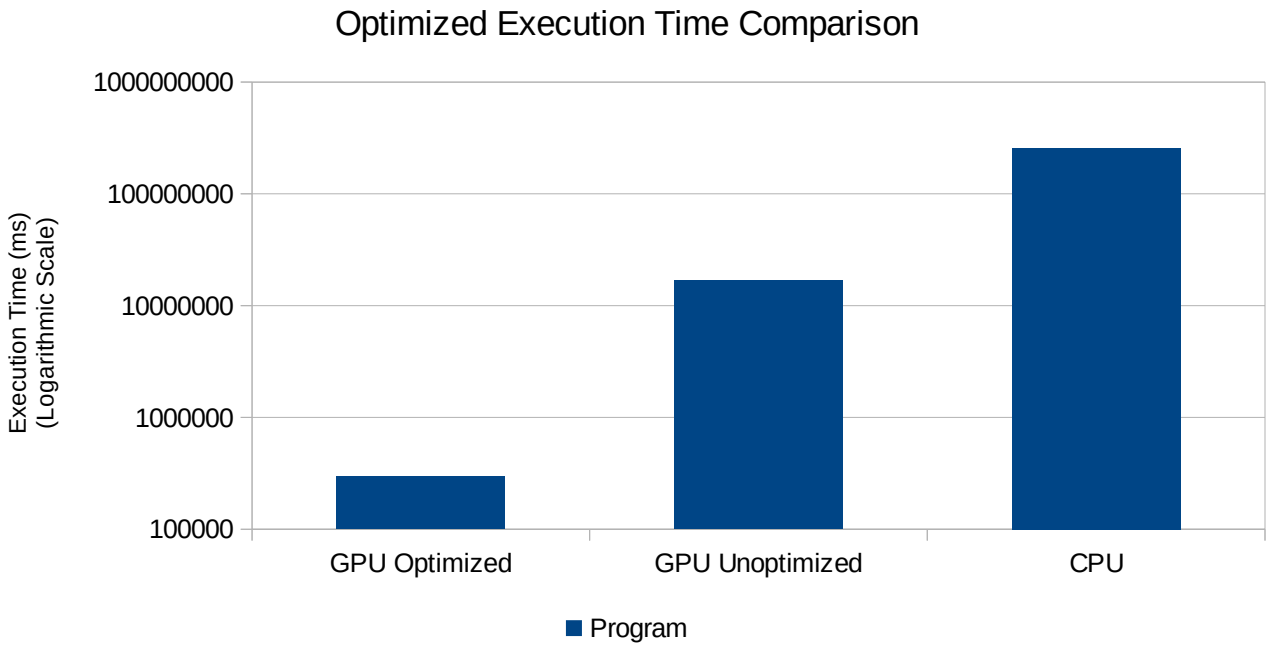


*Figure 5: All execution times were measured using std:chrono at the call of the actual algorithm (i.e. gemmLayer_gpu). Results collected from a (4096 x 4096) * (4096 x 4096) matrix multiply. UVM is was not used for these tests.*

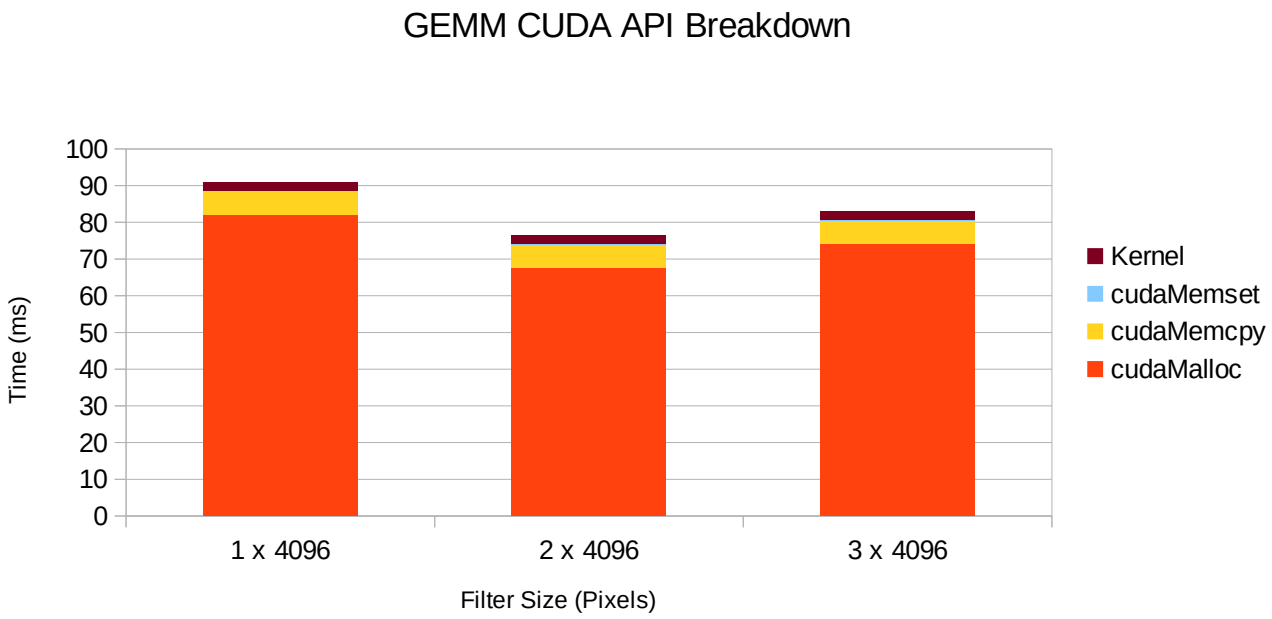


*Figure 6: Breakdown of optimized (without UVM) GEMM API calls using command "nsys nvprof ./lab3" Non-relevant API calls are excluded (i.e. cudaLibraryLoadData) for clarity. Results collected from a (N x 4096) * (4096 x 4096) matrix multiply.*

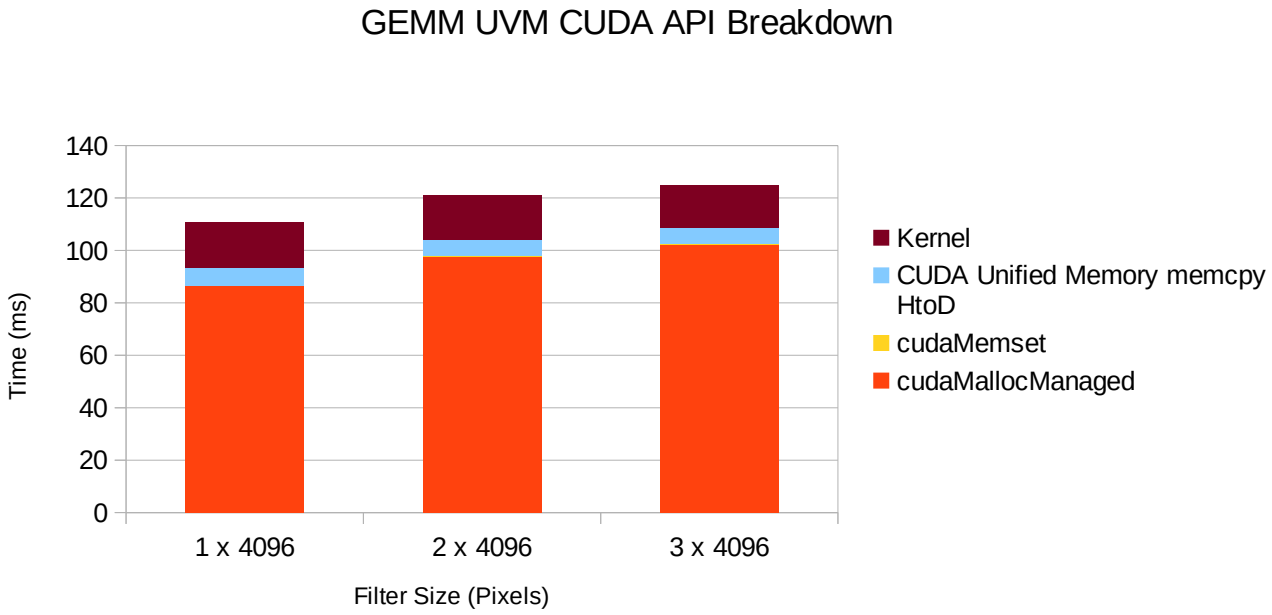


*Figure 7: Breakdown of optimized UVM GEMM API calls using command "nsys nvprof ./lab3" Non-relevant API calls are excluded (i.e. cudaLibraryLoadData) for clarity. Results collected from a (N x 4096) * (4096 x 4096) matrix multiply.*

ECE60827 CUDA Assignment 3
Jerry Liu, 3/25/2023

Observations:

As expected, the CPU version is many times slower than the GPU implementations. Figure 4 suggests that the optimized and unoptimized GPU implementations are nearly identical in performance, but figure 5 shows that the optimized version is indeed much faster. The reason the first table showed the two implementations being nearly equivalent is because the A matrix size was too “skinny” to take advantage of the parallel subtile computation; a more “fat” 4096 x 4096 matrix shows the optimization's true performance benefit.

The data also shows that using unified virtual (UVM) is measurably slower than using explicit memory transfers. This is most likely caused by how UVM works, as well as how the execution time is measured. UVM dynamically moves memory between the host and device as it is needed, and since the input tensors needs to be initialized on the host side, the A and B matrices reside in host memory at kernel launch. When the kernel then attempts to access the input matrices, the host must transfer the data to the device before the kernel can access it. Even worse, the access pattern of the kernel may cause additional unnecessary transfers that would be eliminated if the entire matrix was transferred at once. The same will also occur for the output matrix, but to a lesser degree since no reads are necessary.

Total: 100/100

conv layer: 50/50
functionality: 25/25
memory optimizations: 15/15
GPU kernel optimizations: 10/10

GEMM: 50/50
functionality: 20/20
memory optimizations: 15/15
Comparison with UVM: 15/15